The Gromov–Wasserstein problem

Match relationships when features cannot be compared directly. Suppose two datasets describe different collections of objects, or two different measurements of the same objects. Their feature spaces may be incompatible. Gromov–Wasserstein (GW) matching uses the distances *within* each dataset: a good correspondence pairs objects so that their relationships are preserved. It does not require a distance between an object in one feature space and an object in the other.

Data and unknown. Let $A \in \mathbb{R}^{n\times n}$ and $B \in \mathbb{R}^{m\times m}$ be the two symmetric distance matrices. Positive probability vectors a and b specify the mass of each object; uniform weights give $a_i = 1/n$ and $b_j = 1/m$. The unknown is a **soft matching** $P \in \mathbb{R}^{n\times m}$: P_{ij} is the mass assigned between source object i and target object j. Balanced matching requires

$$U(a,b) = \{P \geq 0 : \ P\mathbf{1}_m = a, \ P^\top \mathbf{1}_n = b\}.$$

The row and column sums prescribe how much each object contributes. They do not force a one-to-one matching.

Objective: preserve pairwise distances. The unregularized squared GW problem used here is

$$\boxed{\min_{P\in U(a,b)} f(P)}, \qquad f(P) = \sum_{i,k=1}^{n}\sum_{j,\ell=1}^{m} (A_{ik} - B_{j\ell})^2 P_{ij}P_{k\ell}.$$

Two simultaneous matches, $i \leftrightarrow j$ and $k \leftrightarrow \ell$, incur a cost when A_{ik} and $B_{j\ell}$ disagree. Equivalently, $f(P)$ is the expected squared distance discrepancy from two independent draws from P. This convention includes neither a factor $1/2$ nor entropy regularization. The objective is smooth and generally nonconvex; the feasible transport polytope is convex. A small loss alone does not certify a global optimum or a scientifically correct correspondence.

Composite form for FraMES. Separate the inexpensive row constraints from the column constraint:

$$C = \{P \geq 0 : P\mathbf{1}_m = a\}, \qquad T(P) = P^\top \mathbf{1}_n,$$

$$g(y) = \iota_{\{b\}}(y) = \begin{cases} 0, & y = b, \\ +\infty, & y \neq b, \end{cases} \qquad \min_{P\in C} f(P) + g(TP).$$

Smoothing the column constraint and its gradient. The indicator g is not differentiable; the smooth quantity is its Moreau envelope. Projection onto the singleton $\{b\}$ gives

$$\operatorname{prox}_{\mu g}(y) = b, \qquad g_\mu(y) = \frac{\|y-b\|_2^2}{2\mu}, \qquad \nabla g_\mu(y) = \frac{y-b}{\mu}.$$

Under Frobenius and Euclidean inner products, $T^*u = \mathbf{1}_n u^\top$. The chain rule therefore gives the explicit penalty gradient

$$\boxed{\nabla(g_\mu \circ T)(P) = \frac{T^*(TP-b)}{\mu} = \frac{\mathbf{1}_n(P^\top\mathbf{1}_n - b)^\top}{\mu}.}$$

This term is computed directly from the current column residual, without random probes. A smaller μ strengthens the penalty for a fixed residual.

FraMES combines a momentum estimate of $\nabla f(P)$ with this current penalty gradient. Its linear minimization oracle places each row's mass in a minimizing column. Convex-combination updates preserve nonnegativity and row masses; column masses can be violated during optimization.

What a results page measures. Feasible loss is the full f evaluated after output-only transport repair to numerical tolerance. Raw loss, feasibility error and the smoothed penalty are separate quantities. Loss magnitudes depend on distance scaling, so reductions are interpreted within a fixed dataset. The next page explains the stochastic oracle for ∇f.

Shared background. Page 2 explains the stochastic oracle; page 3 describes the experiment's data, settings and results.

The stochastic gradient oracle

The target is the gradient of the full GW objective. Let $r = P\mathbf{1}_m$, $c = P^\top \mathbf{1}_n$ and let $\odot 2$ denote entrywise squares. For symmetric A, B,

$$\begin{aligned}\nabla f(P) &= M(P) - 4APB,\\ M(P) &= 2(A^{\odot 2}r)\mathbf{1}_m^\top + 2\mathbf{1}_n(B^{\odot 2}c)^\top.\end{aligned}$$

We compute $M(P)$ directly and estimate APB. The *actual* marginals are used: substituting $c = b$ at a column-infeasible iterate would give the wrong gradient.

A probe is a random projection vector. For each oracle call, draw q fresh, independent vectors $z_s \in \{-1, +1\}^m$, with independent, equiprobable signs in every entry. Stack them into $Z = [z_1, \dots, z_q] \in \mathbb{R}^{m\times q}$. Each vector involves all target points, and multiplication by P and A also involves all source points. Thus q counts projection vectors, not sampled cells or coupling entries. The full coupling is retained with no imposed rank restriction.

The implemented estimator. Using the same Z in both terms, the oracle returns

$$\boxed{\widehat{G}(P; Z) = M(P) - \frac{2}{q}\left[A(P(BZ))Z^\top + A(PZ)(BZ)^\top\right].}$$

The computation forms BZ, PZ and $P(BZ)$, multiplies the last two by A, then forms the two $n \times m$ products above, avoiding explicit construction of APB. Work grows with q; large sketches can cost more than the full gradient.

Why there are two product terms. The estimator is the derivative of the following unbiased scalar sketch of GW:

$$\widehat{f}_Z(P) = r^\top A^{\odot 2} r + c^\top B^{\odot 2} c - \frac{2}{q}\left\langle PZ,\ A(P(BZ))\right\rangle_F.$$

Differentiating both occurrences of P in the inner product produces the two terms in $\widehat{G}$. The matrices A, B and the objective being optimized remain fixed. Reported losses are evaluations of the full f, rather than this random sketch.

Conditional unbiasedness. The probes are independent of the current iterate and satisfy $\mathbb{E}[ZZ^\top/q] = I_m$. Conditional on P,

$$\begin{aligned}\mathbb{E}\Big[A(P(BZ))Z^\top/q \mid P\Big] &= APB,\\ \mathbb{E}\Big[A(PZ)(BZ)^\top/q \mid P\Big] &= APB,\end{aligned} \qquad \mathbb{E}[\widehat{G}(P; Z) \mid P] = \nabla f(P).$$

The second identity uses $B = B^\top$. Fresh probes give the same unbiasedness conditional on the entire optimization history before that draw.

What the probe count controls. At a fixed P, the estimator averages q independent single-probe estimates. If $V(P)$ is the single-probe mean-squared Frobenius error, then

$$\mathbb{E}\Big[\|\widehat{G}(P; Z) - \nabla f(P)\|_F^2 \mid P\Big] = \frac{V(P)}{q}.$$

Four times as many probes halves the root-mean-square error at that same iterate. This does not imply a corresponding improvement in loss or optimization time: the iterates themselves change with q. Even $q \geq m$ does not make independent random-sign probes exact, since $ZZ^\top/q$ need not equal I_m for a finite draw.

Connection to the update. FraMES averages outputs into d_k and uses $H_k = d_k + \nabla(g_{\mu_k} \circ T)(P_k)$. The penalty gradient is computed without sampling; momentum is separate from the oracle.

Page 3 specifies the data, probe count, numerical precision and experimental results.

Synthetic data: increasing the probe count

Experiment results / verified 11 October 2026 12 of 12 runs completed

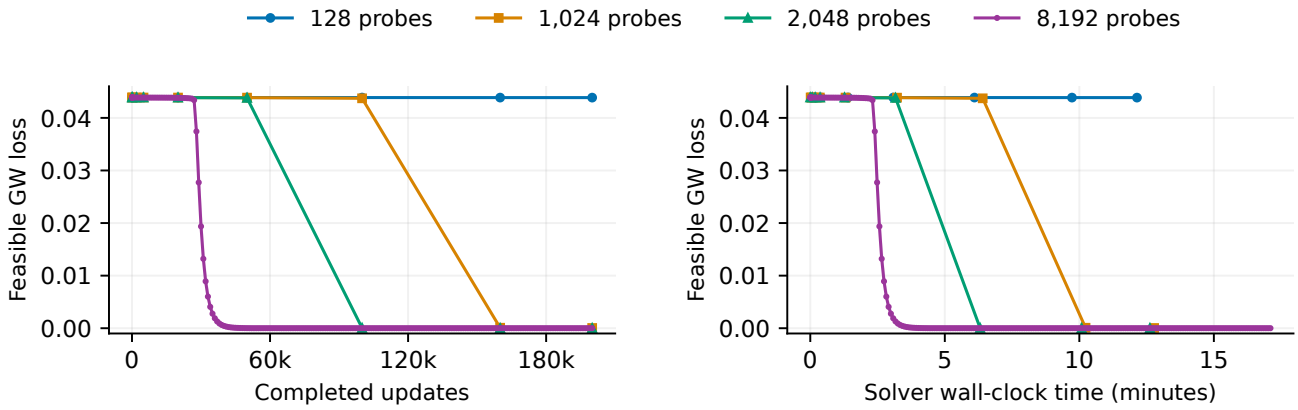
Data. Two 512×512 distance matrices, giving 262,144 coupling variables. The source is a symmetric random metric with off-diagonal distances uniform in $[0.5, 1)$ and zero diagonal; the target is an exact permutation of it (seed 1000, no noise or rescaling). Uniform masses are used. The planted matching has **GW loss zero**, but the optimizer is not given that matching.

Design. Stochastic FraMES with 128, 1,024, 2,048 or 8,192 random-sign probes per gradient; three development seeds (710–712), 200,000 updates each, product start $P_0 = ab^\top$, and one Ruche A100 per run. One gradient estimate is formed per update, with $d_0 = \widehat{G}_0$. All other settings are fixed across probe counts:

$$\gamma_k = (k+1)^{-3/4}, \qquad \rho_k = 0.3(k+1)^{-1/2} \ (k \geq 1), \qquad \mu_k = \frac{10}{512}(k+1)^{-1/4}.$$

Probes	Final feasible loss ↓	Decrease from P_0	Solver minutes
128	0.04385652	0.0616%	12.14
1,024	0.00004589	99.8954%	12.78
2,048	0.00001531	99.9651%	12.62
8,192	0.00000896	99.9796%	17.10

Medians across three seeds; initial feasible loss $f(P_0) = 0.04388356$. Percentage decreases are computed per seed before taking the median.



Markers are saved checkpoints; lines are visual guides. Shading shows the three-seed IQR. The time plot places each common-iteration median loss at its median solver time; it does not interpolate to a common time grid.

Assessment. All three 8,192-probe runs first show at least 99% loss reduction at 40,000–42,000 updates, versus a 50,000–100,000 checkpoint bracket with 2,048. At 200,000 updates, the median final loss is 41.5% lower, while solver time rises from 12.6 to 17.1 minutes. The higher probe count brings the loss drop forward; 128 probes still makes little progress.

Scope. The 8,192-probe runs record loss every 1,000 updates, with additional early checkpoints. Smoothing, prefactors, seeds, start and horizon are unchanged; GPU trajectory-parity checks passed for the added diagnostics. This supports a probe-count effect on this clean instance, without establishing that initial smoothing caused the delay or predicting performance on real data.

Measurement. Final-iterate loss after transport repair, evaluated in FP64; initial and intermediate losses use FP32. This is neither a best-checkpoint loss nor a theorem-selected output. Solver time includes initialization and updates; it excludes preparation, repairs, loss diagnostics, logging, snapshots and final FP64 validation. It is not end-to-end elapsed time.

Provenance. Frozen instance `metric512_s1000`; run sources `8d11e2a` (128–2,048 probes) and `49911ec` (8,192). Validated report SHA256 prefixes: `6ce0b0198875649d7` and `969f9f3b225c832a`.